

AI Assistant Coding

Lab - 3.1

Question 1:

Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

- Record the AI-generated code.
- Test the code with multiple inputs.
- Identify any logical errors or missing edge-case handling.

Prompt :

```
#write a program to generate a python function that checks whether a  
given number is palindrome
```

Code :

```
#write a program to generate a python function that checks whether a  
given number is palindrome  
def is_palindrome(number):  
    num_str = str(number)  
    return num_str == num_str[::-1]  
#input number  
number = 121  
if is_palindrome(number):  
    print(f"{number} is a palindrome.")  
else:  
    print(f"{number} is not a palindrome.")
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistants\dasar\AI-Assistant-Coding\Lab2.py"  
s\dasar\AI-Assistant-Coding\Lab2.py"  
s\dasar\AI-Assistant-Coding\Lab2.py"  
121 is a palindrome.
```

Analysis :

The generated code correctly checks palindrome numbers by converting the number to a string and reversing it. However, it does not explicitly handle negative numbers or validate non-integer inputs.

Question 2:

One-Shot Prompting (Factorial Calculation)

Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Example:

Input: 5 → Output: 120

Task:

- Compare the generated code with a zero-shot solution.
- Examine improvements in clarity and correctness.

Prompt :

```
#write a program to compute factorial of a given number
```

Code :

```
#write a program to compute factorial of a given number
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
#input number
number = 4
result = factorial(number)
print(f"The factorial of {number} is {result}.")
#write code for this example input : 5 output : 120
#input number
number = 5
result = factorial(number)
print(f"The factorial of {number} is {result}.")
```

Output :

```
-Coding\Lab2.py"
The factorial of 4 is 24.
PS C:\Users\dasar\AI-Assistant-Coding>
```

Analysis :

The one-shot prompt improves clarity by providing an example ($5 \rightarrow 120$), making the expected behavior explicit. The generated code is correct and clearer than a zero-shot solution, though it still lacks input validation for negative numbers.

Question 3:

Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Examples:

- Input: 153 $\rightarrow$ Output: Armstrong Number
- Input: 370 $\rightarrow$ Output: Armstrong Number
- Input: 123 $\rightarrow$ Output: Not an Armstrong Number

Task:

- Analyze how multiple examples influence code structure and accuracy.
- Test the function with boundary values and invalid inputs.

Prompt :

```
#write program to check weather the given number is armstrong or not
write for example 1 : 153 output : true
```

Code :

```
#write program to check weather the given number is armstrong or not
write for example 1 : 153 output : true
def is_armstrong_number(number):
    num_str = str(number)
    num_digits = len(num_str)
    sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)
    return sum_of_powers == number
#input number
```

```

number = 153
if is_armstrong_number(number):
    print(f"{number} is an Armstrong number.")
else:
    print(f"{number} is not an Armstrong number.")

#now write code for example 2 : 123 output : False
number = 123
if is_armstrong_number(number):
    print(f"{number} is an Armstrong number.")
else:
    print(f"{number} is not an Armstrong number.")

```

Output :

```

153 is an Armstrong number.
123 is not an Armstrong number.
PS C:\Users\dasar\AI-Assistant-Coding>

```

Analysis :

Multiple examples in few-shot prompting improve accuracy by guiding the AI to correctly compute digit powers and compare their sum with the original number.

The code works for valid positive integers but does not handle negative numbers or non-integer inputs.

Question 4:

Context-Managed Prompting (Optimized Number Classification)

Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.
- Compare the output with earlier prompting strategies.

Prompt :

```

#generate an optimized python program that classifies a number as prime
composite neither

```

Code :

```
#generate an optimized python program that classifies a number as prime
composite neither
def classify_number(number):
    if number <= 1:
        return "neither"
    for i in range(2, int(number**0.5) + 1):
        if number % i == 0:
            return "composite"
    return "prime"
#input number take dynamic input from user
number = int(input("Enter a number: "))
classification = classify_number(number)
print(f"{number} is {classification}.")
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab2.py"
Enter a number: 123
123 is composite.
PS C:\Users\dasar\AI-Assistant-Coding> █
```

Analysis :

The context-managed prompt produces an efficient and well-structured solution using $\sqrt{n}$ optimization and proper classification for edge cases (≤ 1). Compared to earlier prompting strategies, it yields clearer logic, better performance, and explicit input handling.

Question 5:

Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

- Record the AI-generated code.
- Test the program with multiple inputs.
- Identify any missing conditions or inefficiencies in the logic.

Prompt :

```
#write a program to check whether that number is a perfect number or
not
```

Code :

```
#write a program to check whether that number is a perfect number or not
def is_perfect_number(number):
    if number < 1:
        return False
    divisors_sum = sum(i for i in range(1, number) if number % i == 0)
    return divisors_sum == number
#input number
number = int(input("Enter a number: "))
if is_perfect_number(number):
    print(f"{number} is a perfect number.")
else:
    print(f"{number} is not a perfect number.")
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab2.py"
Enter a number: 8126
8126 is not a perfect number.
PS C:\Users\dasar\AI-Assistant-Coding> █
```

Analysis :

The zero-shot prompt generates correct logic for checking perfect numbers and works for standard inputs like 6 and 28. However, the approach is inefficient ($O(n)$) and lacks optimization using divisor pairs or $\sqrt{n}$ traversal.

Question 6:

Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

- Input: 8 → Output: Even
- Input: 15 → Output: Odd
- Input: 0 → Output: Even

Task:

- Analyze how examples improve input handling and output clarity.
- Test the program with negative numbers and non-integer inputs.

Prompt :

```
#write a program to generate that determines whether a given number is even or odd Input: 8 output: even input : 15 output : odd input : 0 output: even
```

Code :

```
#write a program to generate that determines whether a given number is even or odd Input: 8 output: even input : 15 output : odd input : 0 output: even
def is_even_odd(number):
    if number % 2 == 0:
        return "even"
    else:
        return "odd"
#input number
number = int(input("Enter a number: "))
result = is_even_odd(number)
print(f"{number} is {result}.")
#input : 15
number = 15
result = is_even_odd(number)
print(f"{number} is {result}.")
#input : 0
number = 0
result = is_even_odd(number)
print(f"{number} is {result}.")
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab2.py"
Enter a number: 16
16 is even.
15 is odd.
0 is even.
PS C:\Users\dasar\AI-Assistant-Coding> █
```

Analysis :

Few-shot examples improve clarity by clearly mapping inputs to expected outputs and reinforcing correct even/odd logic, including for zero. However, the program lacks proper input validation for non-integer values and relies on direct integer conversion, which may raise errors.